

Ask anything

In digital filter design, **stability means that a filter will produce a bounded, controlled output for any bounded input, preventing the system from oscillating uncontrollably or blowing up toward infinity.** If you feed a finite signal into a stable digital filter, the output will remain finite and eventually decay or settle; an unstable filter will amplify the signal exponentially, quickly causing data overflow, clipping, or system failure. Wikipedia +3

The Mathematical Definition (BIBO Stability)

The primary standard for filter stability is **Bounded-Input, Bounded-Output (BIBO) stability**. Walter Scott, Jr. College o...

- **The Rule:** If the input signal x(n) never exceeds some finite maximum value M, then the output signal y(n) must never exceed some finite maximum value N. Walter Scott, Jr. College o...
- **Impulse Response Condition:** For a Linear Time-Invariant (LTI) filter to be BIBO stable, its impulse response h(n) must be **absolutely summable**:

$$\sum_{n=-\infty}^{\infty} |h(n)| < \infty$$

Practically, this means the filter's response to a single spike must eventually die out and approach zero as time goes to infinity. DSPRelated.com +2

The "Unit Circle" Test (z-Plane)

In digital signal processing (DSP), a filter's behavior is analyzed in the complex z-plane using its transfer function H(z). Stability depends entirely on the location of the filter's

Response) outputs. coefficients are too aggressive.

For a causal IIR filter to be stable, **all of its poles must lie strictly inside the unit circle** in the z-plane. YouTube · Electrical Eng... +1

- **Inside the Unit Circle (|z| < 1):** Stable. The feedback decays over time.
- **On the Unit Circle (|z| = 1):** Marginally stable. The filter will oscillate indefinitely (like a perfect sine wave oscillator).
- **Outside the Unit Circle (|z| > 1):** Unstable. The feedback amplifies itself exponentially. Wikipedia +2

Real-World Hazards: Why Filters Unexpectedly Blow Up

Even if you design a filter with poles perfectly inside the unit circle on paper, it can become unstable in practice due to hardware constraints:

1. **Finite Wordlength Effects (Coefficient Quantization):** In a computer or DSP chip, numbers are stored with limited bit-precision. Rounding a filter coefficient slightly can shift a pole's position. If a pole is designed too close to the edge ($|z| \approx 0.99$), quantization error might push it outside the unit circle, causing the filter to explode. Signal Processing Stack E...
2. **Limit Cycles:** Due to rounding errors in the multiplication hardware, a stable IIR filter processing a constant or zero input can get trapped in a non-zero, self-sustaining periodic oscillation.

How Designers Ensure Stability

- Use **FIR filters** whenever computational budgets allow, bypassing stability worries entirely.
- Break high-order IIR filters into a series of **Second-Order Sections (SOS)** or biquads.

Walter Scott, Jr. College of Engine...

Digital Controls & Digital Filters Lectures 19 & 20

Stability Analysis of Digital Control Systems. Digital Filter Design. Stability Analysis (Chapter 7). Definition 1: Bounded Input,

Signal Processing Stack Exchange

matlab - Question of Digital filter; stability and order

Jul 26, 2018 — An IIR filter 's stability is related to the location of its poles. If the region of convergence contains the unit circle, the fil...

DSPRelated.com

Stability Revisited | Introduction to Digital Filters

A filter is stable if its impulse response decays to zero as n goes to infinity. A necessary and sufficient condition for stabil...

Show all

I need to now create a high-pass filter based on Pan and Tompkins transfer function equation (-1/32 + z^-16 -z^-17 + ...

Implementing a high-pass filter based on the **Pan-Tompkins transfer function** on an ESP32 using the Espressif ESP-DSP library requires addressing sample rate scaling, unity gain, and numerical drift.

Here are the specific architectural recommendations and an optimized implementation strategy.

Hz. ResearchGate

Recommendation: Keep the 16-sample structure exactly as-is. A cutoff of 5.38 Hz is completely acceptable for ECG QRS detection (which targets a passband of 5–15 Hz). Altering the delays to match 5.0 Hz perfectly would require fractional-delay filters, which destroys the elegant, ultra-fast integer nature of the Pan-Tompkins algorithm. Medium

2. The Unity Gain Disambiguation

In classic literature, the Pan-Tompkins filter is often multiplied by 32 to eliminate fractions for old 8-bit microcontrollers. Because you explicitly wrote the division into your transfer function (`-1/32` and `z^-32/32`), **your equation already has a unity gain of 1 at higher frequencies** and exactly 0 gain at DC. No additional scaling factor is required. ResearchGate +1

3. ESP-DSP Implementation Strategy

Although the transfer function looks like an IIR filter because of the denominator, it is mathematically a **Finite Impulse Response (FIR) filter in disguise**. The pole at perfectly cancels out a zero at in the numerator. Chegg

If you analyze the algebra by polynomial division, the filter expands directly to:

This reveals that the Pan-Tompkins high-pass filter is simply a **16-sample delayed signal minus a 32-point moving average**!

Why you shouldn't use generic IIR/FIR library functions:

1. **Generic IIR (`dsps_biquad_f32`):** Implementing the recursive difference equation in floating-point can introduce a slow numerical drift over time due to the pole sitting exactly on the unit circle ( ).

Below is an efficient, drift-free implementation utilizing a single 32-element ring buffer. You can run this directly inside your ESP32 processing task:

C

```c
#include <stdint.h>

#define BUFFER_SIZE 32
#define DELAY_TAPS  16

typedef struct {
    int32_t history[BUFFER_SIZE];
    int head;
    int32_t sum;
} pan_tompkins_hp_t;

// Initialize the filter state
void pan_tompkins_hp_init(pan_tompkins_hp_t *filter) {
    for (int i = 0; i < BUFFER_SIZE; i++) {
        filter->history[i] = 0;
    }
    filter->head = 0;
    filter->sum = 0;
}

// Process a single sample (Optimized Integer Math)
int32_t pan_tompkins_hp_process(pan_tompkins_hp_t *filter, int32_t x_in) {
    // 1. Get the oldest sample out of the 32-sample window
    int32_t x_old = filter->history[filter->head];

    // 2. Update the moving average summation window
    filter->sum = filter->sum + x_in - x_old;

    // 3. Overwrite the oldest sample with the new incoming sample
    filter->history[filter->head] = x_in;
```

```
    return y_out;
}
```

Use code with caution.

Why this approach excels on the ESP32:

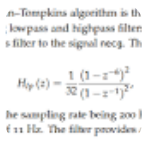
- **Zero Floating-Point Drift:** By maintaining the moving average summation (`filter->sum`) in integers, rounding errors cannot accumulate.
- **No Multiplications:** The division by 32 is handled via a bitwise right-shift (`>> 5`), which compiles down to a single-cycle instruction on both Xtensa and RISC-V ESP32 cores.
- **Low Cache Footprint:** The entire structure takes less than 150 bytes of RAM, keeping it entirely within the ESP32's fast L1 data cache.

Are you planning to process the raw **16-bit ADC values** directly as integers, or do you need to convert them to **voltages (floats)** before filtering? If you are moving to the next steps of the Pan-Tompkins pipeline, I can also provide the optimized code for the **5-point derivative** or the **moving window integrator**.

Chegg

The first step in the Pan-Tompkins algorithm is the | Chegg.com

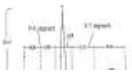
Nov 19, 2020 — After the low-pass filter, the high-pass filter is applie. Together, they form a band-pass filter. A suitable transfer function of...



ResearchGate

Pattern analysis of different ECG signal using Pan-Tompkin's ...

The différence équation is Y (n T) = 32 x (n T – 16T) – [y (n T – T) + x



Can you do both integer and float filtering versions? And yes, please output your recommendations for the derivative and...

Here are **both the integer and floating-point versions of the high-pass filter, followed by the optimized design recommendations and code for the Derivative and Moving Window Integrator stages of the Pan-Tompkins pipeline.**

---

1. High-Pass Filter: Integer vs. Floating-Point

Option A: Integer Version (Recommended for Speed & Stability)

This version processes the raw 16-bit signed integer values directly from the ADS1113. It uses a bit-shift (`>> 5`) instead of division. To prevent negative integer bit-shift issues in C, we cast the sum to a signed integer.

C

```c
#define HP_BUFFER_SIZE 32
#define HP_DELAY_TAPS  16

typedef struct {
    int32_t history[HP_BUFFER_SIZE];
    int head;
    int32_t sum;
} pt_hp_int_t;

void pt_hp_int_init(pt_hp_int_t *f) {
    for (int i = 0; i < HP_BUFFER_SIZE; i++) f->history[i] = 0;
    f->head = 0;
    f->sum = 0;
}
```

```
    // Sum divided by 32 via arithmetic right-shift
    return x_delayed - (f->sum >> 5);
}
```

Use code with caution.

Option B: Floating-Point Version

If you convert your ADC codes to voltages first, use this version. To prevent **floating-point drift** (where tiny rounding errors accumulate over millions of samples and cause the baseline to slowly wander), we use a standard `for` loop to recalculate the sum every time. This avoids accumulation errors.

C

```c
typedef struct {
    float history[HP_BUFFER_SIZE];
    int head;
} pt_hp_float_t;

void pt_hp_float_init(pt_hp_float_t *f) {
    for (int i = 0; i < HP_BUFFER_SIZE; i++) f->history[i] = 0.0f;
    f->head = 0;
}

float pt_hp_float_process(pt_hp_float_t *f, float x_in) {
    f->history[f->head] = x_in;

    int delay_idx = (f->head + HP_DELAY_TAPS) % HP_BUFFER_SIZE;
    float x_delayed = f->history[delay_idx];

    // Explicit sum loop avoids floating-point integration drift
    float sum = 0.0f;
    for (int i = 0; i < HP_BUFFER_SIZE; i++) {
```

2. Derivative Stage

The Pan-Tompkins derivative is a 5-point linear-phase derivative filter. It acts as a differentiator up to about 30 Hz, perfectly highlighting the steep slopes of the QRS complex while suppressing low-frequency baseline wander and high-frequency noise.

- **Original Equation:**
- **Sample Rate Adjustment: Keep this exactly as-is.** Changing your rate from 200 Hz to 215 Hz narrows the bandwidth by a negligible fraction, which has no practical effect on QRS slope detection.
- **Gain:** The gain at 215 Hz is approximately unity for the QRS slopes.

C

```c
typedef struct {
    int32_t history[5]; // Stores x[n] through x[n-4]
    float f_history[5];
} pt_deriv_t;

// Integer Derivative
int32_t pt_deriv_int_process(pt_deriv_t *f, int32_t x_in) {
    // Shift history
    f->history[4] = f->history[3];
    f->history[3] = f->history[2];
    f->history[2] = f->history[1];
    f->history[1] = f->history[0];
    f->history[0] = x_in;

    // y[n] = (2*x[n] + x[n-1] - x[n-3] - 2*x[n-4]) / 8
    int32_t y = (f->history[0] << 1) + f->history[1] - f->history[3] - (f
    return y >> 3;
}
```

Use code with caution.

---

3. Moving Window Integrator (MWI) Stage

After the derivative, the signal is squared ( ) to make all data points positive. The MWI then smooths the squared signal to extract the envelope of the QRS complex.

- **Sample Rate Adjustment Required: Yes.** The window size of the integrator dictates how long a QRS complex is assumed to last. In the original 200 Hz paper, a **30-sample window** was used, which translates to exactly **150 milliseconds**.
- **New Window Calculation:** At 215 Hz, to maintain an identical 150 ms window, we need:

- **Design Advantage:** Adjusting the window size to **exactly 32 samples** makes the integer implementation incredibly efficient, as dividing the moving average sum can be handled via a fast bit-shift (`>> 5`).

C

```c
#define MWI_BUFFER_SIZE 32

typedef struct {
    int64_t history[MWI_BUFFER_SIZE]; // Use 64-bit to prevent overflow du
    int head;
    int64_t sum;
```

```
    f->head = (f->head + 1) % MWI_BUFFER_SIZE;

    // 3. Average over 32 samples (using fast shift)
    return (int32_t)(f->sum >> 5);
}

// Floating-Point Integrator
float pt_mwi_float_process(pt_mwi_t *f, float x_in) {
    float squared = x_in * x_in;
    f->f_history[f->head] = squared;

    float sum = 0.0f;
    for (int i = 0; i < MWI_BUFFER_SIZE; i++) {
        sum += f->f_history[i];
    }

    f->head = (f->head + 1) % MWI_BUFFER_SIZE;

    return sum / 32.0f;
}
```

Use code with caution.

Recommendation Summary

If your pipeline handles raw ADC counts, use the **Integer versions**. They are immune to the rounding instabilities that can plague floating-point algorithms over long runtime windows, and they completely avoid floating-point assembly instructions on the ESP32.

Would you like help combining these stages into a unified **ESP-IDF freeRTOS processing task**, or do you need assistance structuring the final **adaptive thresholding algorithm** to pick out the R-peaks from the MWI output?

hardware of this chip is fundamentally a **16-bit signed (two's complement)** converter. PTSolns +1

When configured for the **4.096V Full-Scale Range (FSR)**, the raw values span from **-32,768 to +32,767**. Because your signal is single-ended (0 to 4.096V), the raw codes outputted by your I2C read will only fall into the positive half of the spectrum: **0 to +32,767**. TI E2E support forums +1

Here is exactly how to pretreat that raw stream for both pipeline architectures.

---

1. Pretreatment for the Integer Version

For the integer pipeline, you do **not** want to map the values to physical voltages. Instead, you feed the raw `int16_t` bits straight into your processing loop. However, you must implement two minor steps to maintain signal integrity and ensure mathematical safety: ODG Electronics

- **Sign-Extension Cast:** Ensure the variable holding your raw 16-bit payload is immediately cast or read into a signed 32-bit integer (`int32_t`). This prevents any chance of unexpected arithmetic overflow during subsequent stages. Adafruit Forums
- **DC Bias Offset (Optional but Recommended):** In its raw state, your ECG signal sits on a large DC mountain (~2V baseline a raw value of +16,000). While the Pan-Tompkins high-pass filter is designed to track and strip DC baselines, feeding a giant 16,000-count offset right into an empty, zero-initialized filter buffer can cause a massive settling transient spike that takes 32 samples to decay. Subtracting a fixed midpoint helps your pipeline settle instantly on startup. Adafruit Forums

Integer Pretreatment Code:

C

```
// Incoming variable from your I2C read function
```

Use code with caution.

2. Pretreatment for the Floating-Point Version

For the floating-point version, you must convert the raw counts into true physical voltages. ODG Electronics

At the 4.096V range setting, the Least Significant Bit (LSB) size is exactly **125 microvolts per step** ( ). Multiplying your raw code by this constant (`0.000125f`) translates your signal into a true voltage float between `0.0f` and `4.096f` . ODG Electronics

Floating-Point Pretreatment Code:

C

```c
// Incoming variable from your I2C read function
int16_t raw_adc_data = read_ads1115_register();

// 1. Convert raw counts to physical floating-point volts (1 LSB = 125 uV)
// 0 counts = 0.0V, 32767 counts = 4.095875V
float x_in_volts = (float)raw_adc_data * 0.000125f;

// 2. Pass the voltage to your floating-point filter pipeline
float hp_out = pt_hp_float_process(&my_hp_filter, x_in_volts);
```

Use code with caution.

Why the Integer Version Wins:
A raw integer slope of `400 counts` squared becomes `160,000 counts`. This massive headroom makes peak detection thresholds drastically easier to compute and completely removes any threat of floating-point underflow or precision loss.

Would you like help mapping out the **ESP-DSP memory configurations** for the processing buffers, or are you ready to look at how to implement the final **R-peak thresholding logic**?

ODG Electronics

Never Misread a Sensor Again with ADS1115 | ODG - Origin-IC

Oct 27, 2025 — CONVERTING RAW VALUES TO VOLTAGE The readADC_SingleEnded() function returns a raw integer value. For the 16-bit ADS1115, this numb...



Adafruit

Problem with ADS1115 16 bit ADC

Jan 2, 2013 — Here's some information about the ADS1115 16 bit ADC: * The 16 bit output is a signed value. * When used in single-ended mode, the...

Adafruit Forums

Advice on how to get 16 bits from ADS1115 with a potentiomet

Sep 5, 2020 — Re: Advice on how to get 16 bits from ADS1115 with a potenti ... The 16th bit is the positive or negative sign on the 15-bit value...

Show all

You purposely steered away from ESP-DSP library. I do like your suggestions, but what specifically are the drawbacks ...

include:

1. Block-Based Processing vs. Sample-by-Sample Streaming

- **The ESP-DSP Design:** ESP-DSP achieves its blazing-fast speeds by leveraging the ESP32's vector instructions (SIMD). To do this, functions like `dsps_fir_f32` or `dsps_biquad_f32` require a **block of data** (an array of samples) to process at once.
- **The Pan-Tompkins Reality:** You are reading samples from an ADS1115 at **215 Hz via I2C**. Your data arrives sequentially, one single sample every **4.65 milliseconds**. If you try to call an ESP-DSP library function for just *one sample* at a time, the function's internal overhead (setting up loop counters, loading pointers into assembly registers) completely wipes out the optimization. Alternatively, if you buffer data into blocks of 32 or 64 samples to make ESP-DSP happy, you introduce **150 to 300 milliseconds of artificial processing latency**, delaying your real-time R-peak detection.

2. Computational Inefficiency (32 Multiplies vs. 0 Multiplies)

If you implemented the Pan-Tompkins high-pass or integrator using ESP-DSP's FIR function (`dsps_fir_f32`):

- **ESP-DSP** would store an array of 32 floating-point coefficients (all equal to `0.03125f` for the moving average). For every single new sample, the ESP32 would run an assembly loop executing **32 floating-point multiplications and 32 additions**.
- **The Ring Buffer Approach** recognizes that a moving average only requires you to add the new sample and subtract the oldest sample. It performs exactly **1 addition, 1 subtraction, and 1 bit-shift**, requiring **zero multiplications**.
- Even with ESP-DSP's assembly optimizations, 64 floating-point operations will always be significantly slower and consume more power than 2 integer operations.

3. Numerical Precision and Drift on the Unit Circle

As discussed in the stability overview, the Pan-Tompkins high-pass filter has a pole

4. Data Type Mismatches

ESP-DSP is heavily biased toward **32-bit floating-point (`f32`)** or **16-bit fixed-point fractional math (`q15`)**. Your ADS1115 outputs raw 16-bit signed two's complement integers (`int16_t`).

- If you use the `f32` library, you are forced to spend CPU cycles converting integers to floats at the input, and then you lose precision down the line when you square the values.
- If you use the `q15` library, you have to carefully scale your data to sit strictly between `-1.0` and `+1.0`, introducing significant risk of arithmetic underflow or clipping during the squaring and integration stages.

When *Should* You Use ESP-DSP for ECG?

You should absolutely use ESP-DSP if you decide to abandon the classic 1985 Pan-Tompkins integer shortcut and instead design a modern, high-order **Butterworth or Chebyshev IIR filter** to handle your baseline wander and powerline noise. For those, `dsps_biquad_f32` combined with their Second-Order Sections (SOS) structures is beautifully optimized, perfectly stable, and exactly what the library was built for.

Knowing these architectural tradeoffs, would you prefer to stick with the **ultra-lightweight custom loops** tailored for the Pan-Tompkins algorithm, or would you like to pivot to using **ESP-DSP's biquad functions** to build a more modern, mathematically rigorous ECG filtering pipeline?

one point I would make in reference to 1.: If I were to utilize a ping-pong buffering scheme so that, when I filled "ping", I

A ping-pong buffer ensures your CPU can process data in "Ping" while your ADC ISR/task is filling up "Pong" without memory collisions. However, **a buffer cannot be processed until it is full.**

Let's look at the math for your **215 Hz sample rate** using a relatively small block size of **32 samples** to make ESP-DSP happy:

1. Sample 1 arrives at time t = 0 ms. It sits in the "Ping" buffer.
2. Sample 2 arrives at t = 4.65 ms.
3. ...
4. Sample 32 arrives at t = 148.8 ms.

Only *now* is the "Ping" buffer full. Your system flips the incoming data to "Pong" and instantly releases "Ping" to the ESP-DSP library.

Even if ESP-DSP processes that entire block in a blazing-fast **0.01 milliseconds**, Sample 1 has already been delayed in RAM for **148.8 milliseconds** before the filter ever sees it.

Why This Matters for ECG

In a clinical monitor, a **150 ms delay** is usually acceptable. However, for real-time applications (such as triggering an external pacemaker, syncing a defibrillator, or firing an immediate audible alarm), standard medical guidelines usually require an end-to-end latency of **under 50 to 100 ms**.

When you add the 148.8 ms block delay to the high-pass filter's inherent **74.4 ms group delay** (the 16-sample delay we discussed earlier), your R-peak detection will lag behind the user's actual heart ventricles by over **220 milliseconds**.

If You Still Want to Use ESP-DSP: The Correct Way

```
#define BLOCK_SIZE 32
#define FIR_TAPS   33 // A 32-delay filter requires 33 coefficients (h[0]

// Filter state and coefficients
float fir_coeffs[FIR_TAPS];
float fir_delay_line[FIR_TAPS];
fir_f32_t hp_fir;

void init_espdsp_highpass() {
    // 1. Manually calculate the exact impulse response of Pan-Tompkins
    // y[n] = x[n-16] - (1/32) * sum_{k=0}^{31} x[n-k]
    for (int i = 0; i < FIR_TAPS; i++) {
        fir_coeffs[i] = 0.0f;
        if (i < 32) {
            fir_coeffs[i] -= (1.0f / 32.0f); // The moving average compone
        }
        if (i == 16) {
            fir_coeffs[i] += 1.0f;           // The center delayed sample
        }
    }

    // 2. Initialize the ESP-DSP FIR structure
    dsps_fir_f32_init(&hp_fir, fir_coeffs, fir_delay_line, FIR_TAPS);
}

// Inside your Ping-Pong processing task
void process_buffer_with_espdsp(float *input_block, float *output_block) {
    // This executes assembly-optimized SIMD matrix multiplications over t
    dsps_fir_f32(&hp_fir, input_block, output_block, BLOCK_SIZE);
}
```

Use code with caution.

The Verdict

stick to sample-by-sample.